# Study case 6: Smart Public Transport Fare Reconciliation

**1. Problem Statement**

In public transport buses, general passengers and students have different fares.

This program calculates the expected amount based on passenger count and fare. Then it compares it with the actual collected amount to find whether there is a shortage, excess, or no difference.

**2. Objectives**

1.To calculate fare from general and student passengers.

2.To calculate expected revenue for each route.

3.To compare expected revenue with collected revenue.

4.To find shortage, excess, or no difference.

5.To calculate total revenue for all routes.

**3.Assumptions**

1.There is at least one bus route.

2.General passengers and students have different fares.

3.Passenger count cannot be negative.

4.The fare entered by the user is correct.

5.The collected amount entered by the user is correct.

6.All amounts are in rupees.

**4.Algorithm**

1.Start the program.

2.Enter the number of bus routes.

3.Enter the route name.

4.Enter general fare and student fare.

5.Enter general and student passenger counts.

6.Calculate expected revenue.

7.Enter the collected revenue.

8.Calculate the difference between collected and expected revenue.

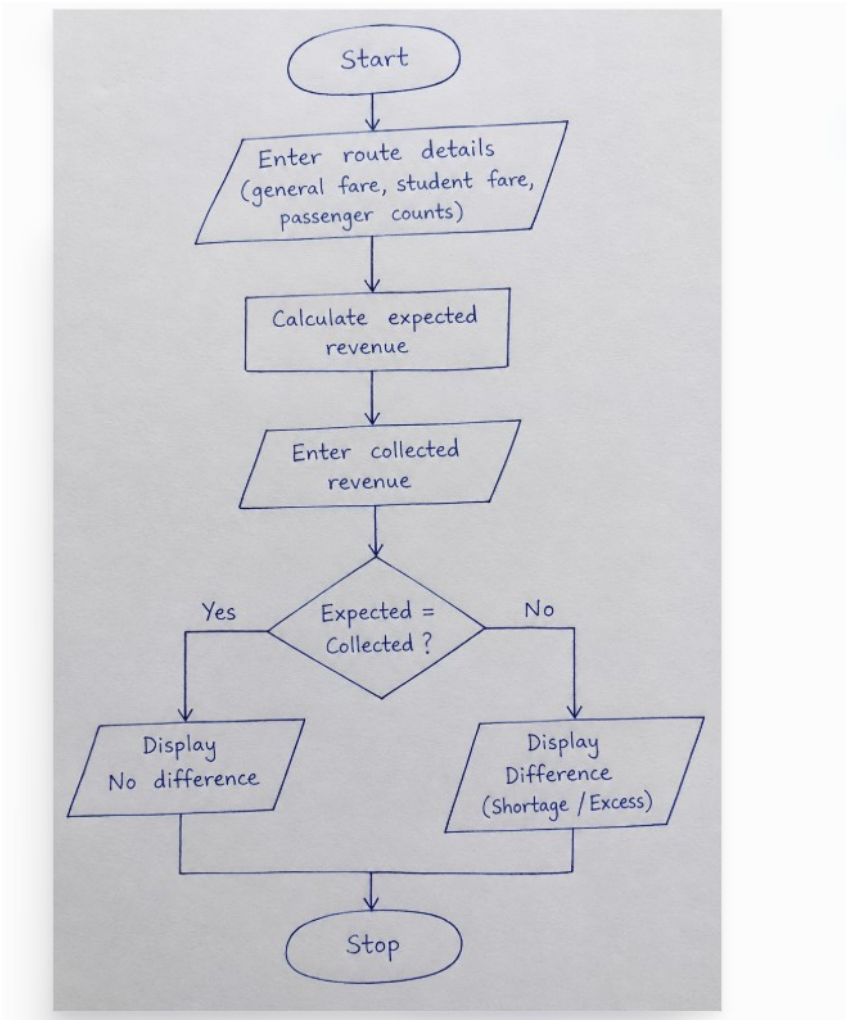
9.Display shortage, excess, or no difference.

10.Repeat the same process for all routes.

11.Display total expected revenue, collected revenue, and difference.

12.Stop.

**5.Flowchart**




**6.Pesudocode**

START

INPUT route name

INPUT general fare

INPUT student fare

INPUT general passengers

INPUT student passengers

CALCULATE expected revenue

INPUT collected revenue

CALCULATE difference

IF expected revenue = collected revenue

  DISPLAY "No difference"

ELSE

  DISPLAY "Difference"

END IF

STOP

**7.Python source code**

```
route = input("Enter route name: ")
general_fare = int(input("Enter general fare: "))
student_fare = int(input("Enter student fare: "))
general = int(input("Enter general passengers: "))
students = int(input("Enter student passengers: "))
expected = (general * general_fare) + (students * student_fare)
collected = int(input("Enter collected revenue: "))
difference = collected - expected
print("Route:", route)
print("Expected revenue:", expected)
print("Collected revenue:", collected)
if difference < 0:
    print("Shortage:", -difference)
elif difference > 0:
    print("Excess:", difference)
else:
    print("No difference")
```

## 8.Test cases

| Test Case | Expected Revenue | Collected Revenue | Result |
|---|---|---|---|
| 1 | ₹1000 | ₹900 | Shortage ₹100 |
| 2 | ₹1000 | ₹1100 | Excess ₹100 |
| 3 | ₹1000 | ₹1000 | No Difference |

## 9.Sample output

```
Enter route name: gannavaram
Enter general fare: 50
Enter student fare: 20
Enter general passengers: 40
Enter student passengers: 1
Enter collected revenue: 41
Route: gannavaram
Expected revenue: 2020
Collected revenue: 41
Shortage: 1979
```

## 10.Conclusion

This program helps to calculate the expected bus fare revenue and compare it with the actual collected revenue. It identifies whether there is a shortage, excess, or no difference. This program uses basic Python concepts like input, variables, calculations, and if-else conditions.

# Study case 7:Examination Result and Grace-Mark Validation

**1. Problem Statement**

Students have marks in different subjects. This program checks whether the student passed or failed in each subject. If the student needs 5 or fewer marks to pass, grace marks can be added according to the rule. Finally, it displays the result.

**2. Objectives**

1. To store marks of different subjects.
2. To check whether the student passed or failed.
3. To add grace marks when required.
4. To make sure marks do not exceed maximum marks.
5. To display the final result.

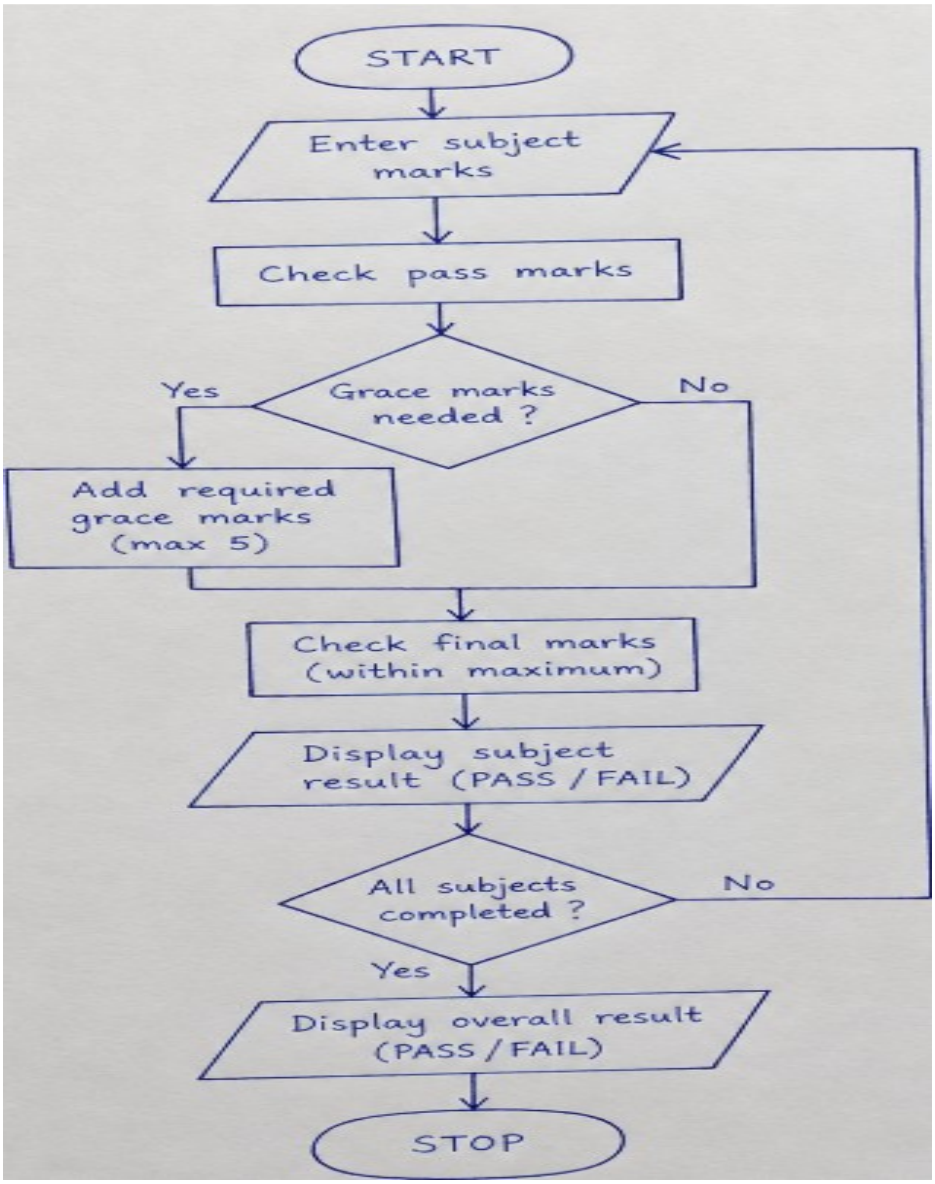
**3. Assumptions**

1. Each subject has obtained marks, maximum marks and pass marks.
2. Maximum 5 grace marks are allowed.
3. Grace marks are given only when the student needs 5 marks or less to pass.
4. Final marks cannot exceed maximum marks.
5. Student passes overall only if all subjects are passed.

**4. Algorithm**

1. Start.
2. Enter the subject marks.
3. Set grace marks limit as 5.
4. Check whether the student passed each subject.
5. If needed marks are 5 or less, add grace marks.
6. Check the final marks.
7. Display PASS or FAIL for each subject.
8. Display the overall result.
9. stop
10. Stop.

**5.Flowchart**

11. 


## 6.Pseudocode

START

INPUT subject marks

IF marks < pass marks

    CHECK grace marks

    IF needed marks <= 5

        ADD grace marks

    END IF

END IF

IF final marks >= pass marks

    DISPLAY "PASS"

ELSE

    DISPLAY "FAIL"

END IF

STOP

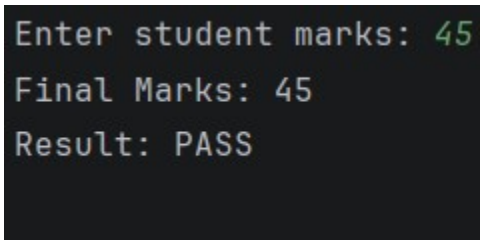
**7.Python source code**

```
marks = int(input("Enter student marks: "))
pass_marks = 40
if marks < pass_marks:
    needed = pass_marks - marks
    if needed <= 5:
        marks = marks + needed
        print("Grace marks added:", needed)
if marks >= pass_marks:
    print("Final Marks:", marks)
    print("Result: PASS")
else:
    print("Final Marks:", marks)
    print("Result: FAIL")
```

**8.Test cases**

| Test Case | Marks | Grace Marks | Final Marks | Result |
|---|---|---|---|---|
| 1 | 45 | 0 | 45 | PASS |
| 2 | 37 | 3 | 40 | PASS |
| 3 | 30 | 0 | 30 | FAIL |

### 9.Sample output




### 10.Conclusion

This program helps to check student marks and apply grace marks when needed. It displays whether the student is PASS or FAIL. This program uses basic Python concepts like variables, if-else conditions, input and output statements.

# Study case 8: Manufacturing Cost and Profitability Analysis

**1. Problem Statement**

To develop a simple Python program to calculate the total manufacturing cost of a product using material cost, labour charges, wastage cost and machine cost. The program calculates the selling price, profit and profit percentage, and checks whether the production meets the specified profit target.

**2. Objectives**

1. Calculate the total material cost.
2. Calculate wastage and labour costs.
3. Add machine operating costs.
4. Calculate total manufacturing cost and profit.
5. Check whether the profit target is achieved.
6. Display the cost and profit details.

**3. Assumptions**

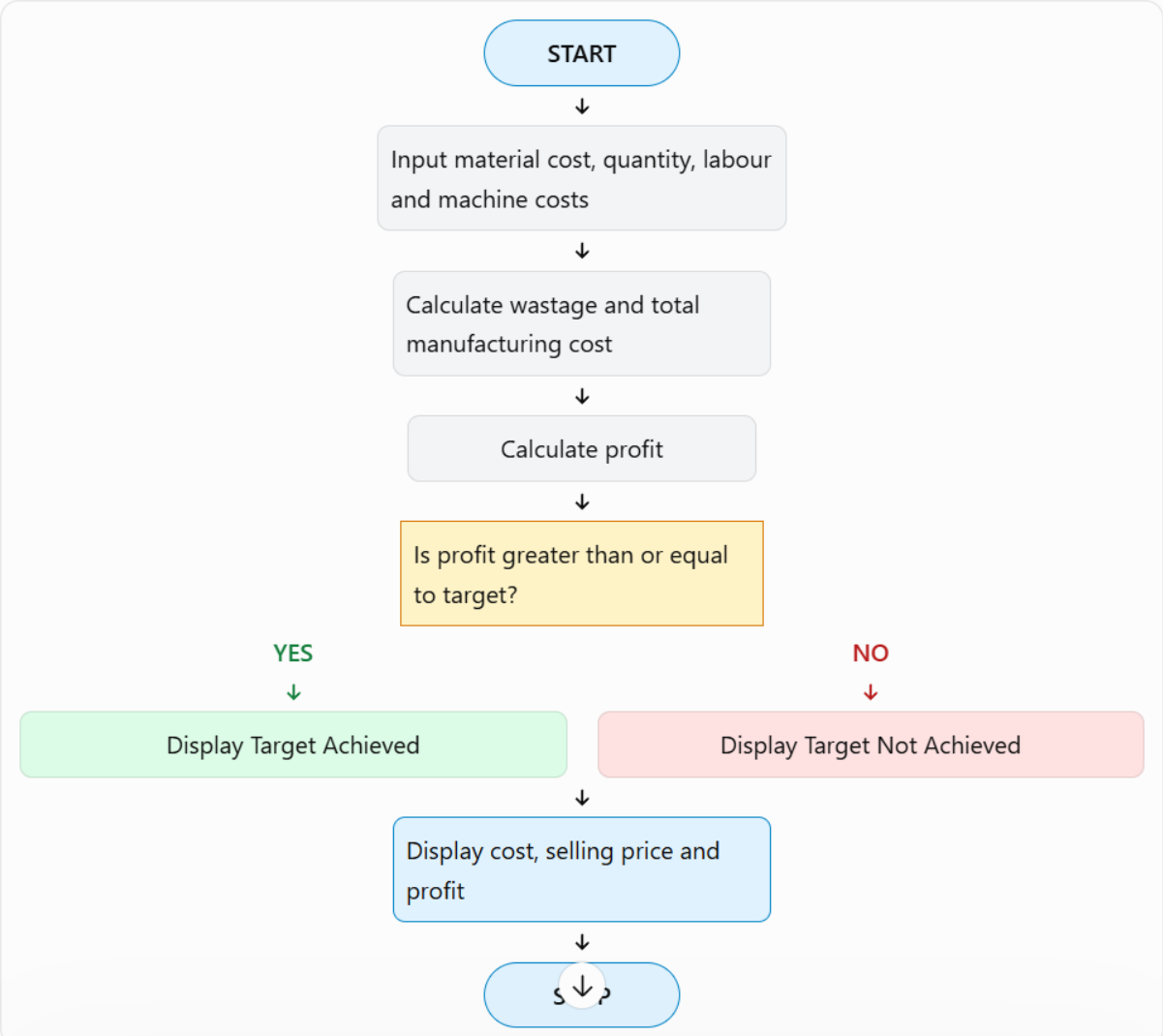
These are sample values for the assignment.

- Material cost is entered by the user.
- Labour charges, wastage cost and machine cost are entered separately.
- Total manufacturing cost is the sum of all four costs.
- Selling price is entered by the user for the total production.
- Profit = Selling price − Total manufacturing cost.
- Profit percentage is calculated on the manufacturing cost.
- The profit target is entered as a percentage.
- All costs and selling price must be non-negative, and manufacturing cost must be greater than zero.

**4. Algorithm**

1. Start the program.
2. Input material cost, labour charges, wastage cost and machine cost.
3. Input the total selling price and target profit percentage.
4. Calculate total manufacturing cost.
5. Calculate profit by subtracting manufacturing cost from selling price.
6. Calculate profit percentage.
7. Check whether the profit percentage meets the target.
8. Display total cost, profit, profit percentage and result.
9. Stop.

**5.Flowchart**



**6.pseudocode**

```
START
Input material cost
Input labour charges
Input wastage cost
Input machine cost
Input selling price
Input target profit percentage
If any cost or selling price is negative:
   Display "Invalid input"
Else:
   Total Cost = Material + Labour
              + Wastage + Machine
   If Total Cost is zero:
      Display "Invalid manufacturing cost"
   Else:
      Profit = Selling Price - Total Cost
      Profit Percentage = (Profit / Total Cost) * 100
      Display Total Cost
      Display Profit
      Display Profit Percentage
      If Profit Percentage >= Target:
```

```
        Display "Target Achieved"
    Else:
        Display "Target Not Achieved"
STOP
```

**7.python source code**

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour charges: "))
wastage = float(input("Enter wastage cost: "))
machine = float(input("Enter machine cost: "))
selling_ price = float(input("Enter total selling price: "))
target = float(input("Enter target profit percentage: "))
if (material < 0 or labour < 0 or wastage < 0
    or machine < 0 or selling_ price < 0
    or target < 0):
  print("Invalid input")
else:
  total_ cost = material + labour + wastage + machine
  if total_ cost == 0:
    print("Invalid manufacturing cost")
  else:
    profit = selling_ price - total_ cost
    percentage = (profit / total_ cost) * 100
    print ("Material Cost: Rs.", material)
    print ("Labour Charges: Rs.", labour)
```

```
print ("Wastage Cost: Rs.", wastage)
print ("Machine Cost: Rs.", machine)
print ("Total Manufacturing Cost: Rs.", total_ cost)
print("Selling Price: Rs.", selling_ price)
print("Profit: Rs.", profit)
print("Profit Percentage:", round(percentage, 2), "%")

if percentage >= target:
    print("Profit Target Achieved")
else:
    print("Profit Target Not Achieved")
```

**8.test cases**

| Test | Total cost | Selling price | Target | Expected result |
|---|---|---|---|---|
| 1 | ₹9,000 | ₹12,000 | 25% | Target achieved |
| 2 | ₹10,000 | ₹11,000 | 15% | Target not achieved |
| 3 | ₹8,000 | ₹8,000 | 10% | Target not achieved |

**9.sample output**

```
Enter material cost: 800
Enter labour charges: 200
Enter wastage cost: 100
Enter machine cost: 200
Enter total selling price: 1000
Enter target profit percentage: 200
Material Cost: Rs. 800.0
Labour Charges: Rs. 200.0
Wastage Cost: Rs. 100.0
Machine Cost: Rs. 200.0
```

```
Total Manufacturing Cost: Rs. 1300.0
Selling Price: Rs. 1000.0
Profit: Rs. -300.0
Profit Percentage: -23.08 %
Profit Target Not Achieved
```

**10.Conclusion**

The Manufacturing Cost and Profitability Analysis program calculates total manufacturing cost, selling price, profit and profit percentage. It checks whether the required profit target is achieved. This program helps first-year B.Tech students understand variables, arithmetic operators, input/output and conditional statements in Python.

# Study case 9: Multi-Currency Travel Budget Analysis

**1. Problem Statement**

To develop a simple Python program that converts travel expenses from foreign currency into Indian Rupees using given exchange rates. The program calculates conversion fees, compares total expenses with the budget, and identifies the largest expense category.

**2. Objectives**

- To convert foreign currency into Indian Rupees.
- To calculate conversion fees.
- To calculate total travel expenses.
- To compare expenses with the planned budget.
- To identify the largest expense category.

**3. Assumptions**

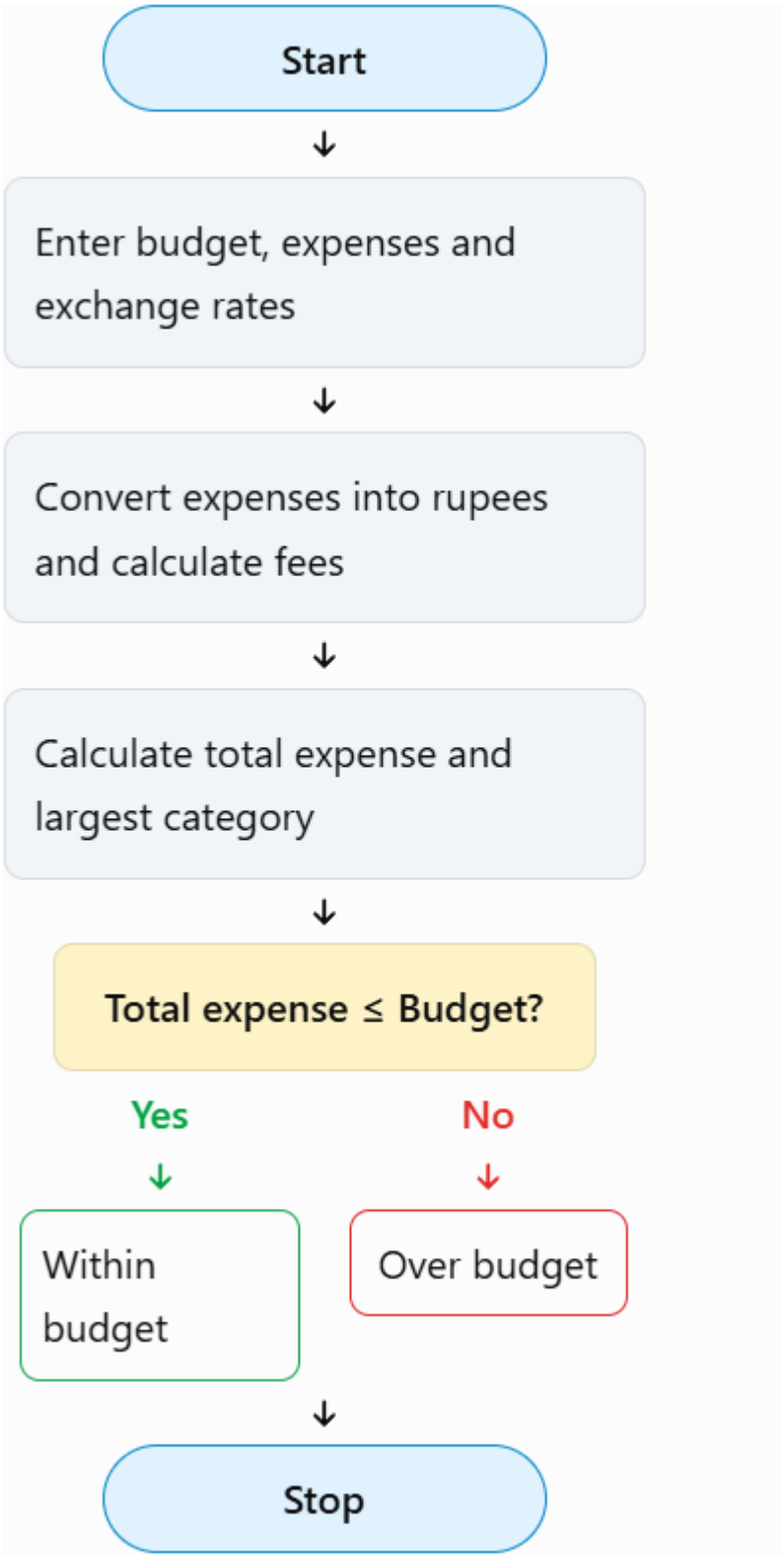
1. Exchange rates are given by the user.
2. Conversion fee is 2% of the converted amount.
3. All expenses are converted into Indian Rupees.
4. Expense amounts and budget must not be negative.
5. The largest expense is identified from the converted amounts, excluding conversion fees.

**4. Algorithm**

1. Start.
2. Enter the travel budget in Indian Rupees.
3. Enter the expenses for food, travel, and shopping in foreign currency.
4. Enter the exchange rate for each expense.
5. Convert each expense into Indian Rupees.

6. Calculate the 2% conversion fee.
7. Calculate the total expense including fees.
8. Identify the largest expense category.
9. Compare the total expense with the budget.
10. Display the results.
11. Stop.

**5.Flowchart**




**6.Pseudocode**

START

INPUT budget

INPUT food expense and exchange rate

INPUT travel expense and exchange rate

INPUT shopping expense and exchange rate

CONVERT each expense into rupees

CALCULATE 2% conversion fee

CALCULATE total expense including fees

FIND the largest expense category

IF total expense <= budget

  PRINT "Within budget"

ELSE

  PRINT "Over budget"

END IF

DISPLAY converted expenses, fees and total

DISPLAY largest expense category

STOP

**7.python source code**

```
print("Multi-Currency Travel Budget Analysis")
budget = float(input("Enter travel budget in rupees: "))
food = float(input("Enter food expense in foreign currency: "))
food_ rate = float(input("Enter food exchange rate: "))
travel = float(input("Enter travel expense in foreign currency: "))
travel_ rate = float(input("Enter travel exchange rate: "))
shopping = float(input("Enter shopping expense in foreign currency: "))
shopping_ rate = float(input("Enter shopping exchange rate: "))
if (budget < 0 or food < 0 or travel < 0
    or shopping < 0 or food_ rate <= 0
    or travel_ rate <= 0 or shopping_ rate <= 0):
```

```
    print("Invalid input")
else:
    food_ rs = food * food_ rate
    travel_ rs = travel * travel_ rate
    shopping_ rs = shopping * shopping_ rate
    fee = (food_ rs + travel_ rs + shopping_ rs) * 0.02
    total = food_ rs + travel_ rs + shopping_ rs + fee
    print("\n--- Travel Expense Report ---")
    print("Food Expense: Rs.", round(food_ rs, 2))
    print("Travel Expense: Rs.", round(travel_ rs, 2))
    print("Shopping Expense: Rs.", round(shopping_ rs, 2))
    print("Conversion Fee: Rs.", round(fee, 2))
    print("Total Expense: Rs.", round(total, 2))
    if food_ rs >= travel_ rs and food_ rs >= shopping_ rs:
        print("Largest Expense: Food")
    elif travel_ rs >= shopping_ rs:
        print("Largest Expense: Travel")
    else:
        print("Largest Expense: Shopping")
    if total <= budget:
        print("Status: Within Budget")
    else:
        print("Status: Over Budget")
        print("Extra Amount: Rs.", round(total - budget, 2))
```

**8.test cases**

| Test | Budget (₹) | Food (₹) | Travel (₹) | Shopping (₹) | Expected result |
|---|---|---|---|---|---|
| 1 | 10,000 | 2,000 | 3,000 | 1,000 | Within budget |
| 2 | 5,000 | 2,000 | 3,000 | 1,000 | Over budget |
| 3 | 8,000 | 1,000 | 2,000 | 4,000 | Within budget |

**9.sample cases**

Multi-Currency Travel Budget Analysis

Enter travel budget in rupees: 10000

Enter food expense in foreign currency: 20

Enter food exchange rate: 100

Enter travel expense in foreign currency: 30

Enter travel exchange rate: 100

Enter shopping expense in foreign currency: 10

Enter shopping exchange rate: 100

--- Travel Expense Report ---

Food Expense: Rs. 2000.0

Travel Expense: Rs. 3000.0

Shopping Expense: Rs. 1000.0

Conversion Fee: Rs. 120.0

Total Expense: Rs. 6120.0

Largest Expense: Travel

Status: Within Budget

**10.Conclusion**

This project helps us understand how to convert foreign currency into Indian Rupees, calculate conversion fees, find the largest expense, and check whether a travel budget is sufficient. It uses basic Python concepts such as variables, arithmetic operators, if-else statements, and input/output functions.

# Study case 10: Campus Event Financial Planning

**1. Problem Statement**

To develop a simple Python program to calculate the total cost of a campus event, including venue, equipment, catering, transport, and certificates. The program also calculates participant contributions, the sponsorship shortfall, and the minimum number of registrations required to break even.

**2. Objectives**

- To calculate the total event cost.
- To calculate the money collected from participants.
- To calculate the sponsorship amount still required.
- To find the minimum registrations needed to cover the event cost.
- To check whether the event budget is sufficient.

**3. Assumptions**

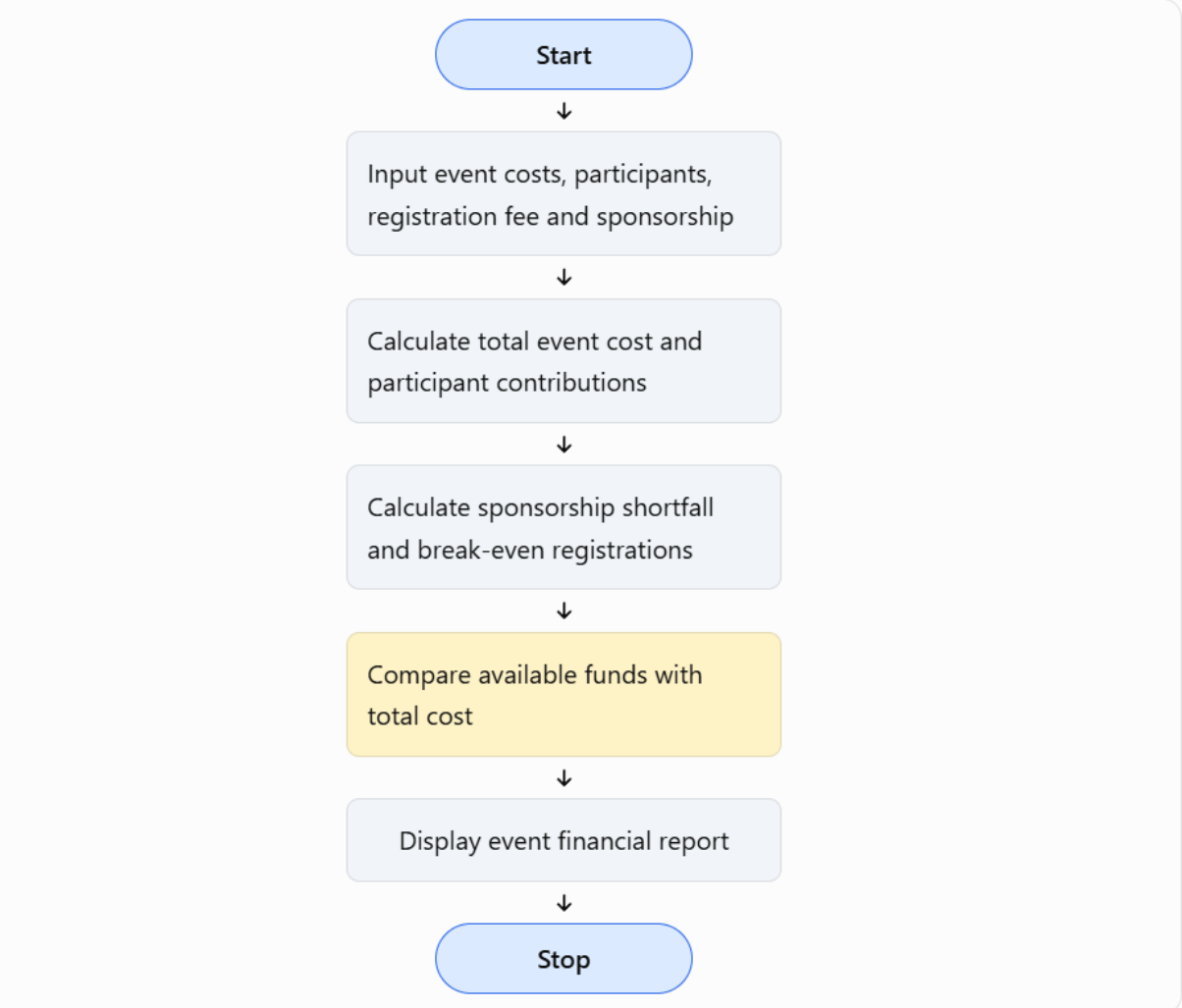
1. Venue and equipment costs are fixed.
2. Catering, transport, and certificate costs are calculated per participant.
3. The participant registration fee is fixed.
4. The sponsorship amount is entered by the user.
5. All costs and amounts must be non-negative.
6. Break-even registrations are calculated using the total cost and registration fee, assuming registration fees alone cover the cost.

**4. Algorithm**

1. Start.
2. Enter venue and equipment costs.
3. Enter catering, transport, and certificate costs per participant.
4. Enter the expected number of participants.

5. Enter the registration fee and sponsorship amount.
6. Calculate the total event cost.
7. Calculate the participant contribution.
8. Calculate the sponsorship shortfall.
9. Calculate the minimum registrations required to break even.
10. Display all results.
11. Stop.

**5.flowchart**



**6.pseudocode**

START

INPUT venue cost

INPUT equipment cost

INPUT catering cost per participant

INPUT transport cost per participant

INPUT certificate cost per participant

INPUT number of participants

INPUT registration fee

INPUT sponsorship amount

CALCULATE total variable cost per participant

CALCULATE total event cost

CALCULATE participant contribution

CALCULATE total available funds

IF available funds < total event cost

    CALCULATE sponsorship shortfall

ELSE

    sponsorship shortfall = 0

END IF

CALCULATE minimum break-even registrations

DISPLAY total event cost

DISPLAY participant contribution

DISPLAY sponsorship shortfall

DISPLAY break-even registrations

STOP

**7.python source code**

```
print("Campus Event Financial Planning")


venue = float(input("Enter venue cost: Rs. "))
equipment = float(input("Enter equipment cost: Rs. "))
catering = float(input("Enter catering cost per participant: Rs. "))
transport = float(input("Enter transport cost per participant: Rs. "))
certificate = float(input("Enter certificate cost per participant: Rs. "))
participants = int(input("Enter number of participants: "))
fee = float(input("Enter registration fee per participant: Rs. "))
sponsorship = float(input("Enter sponsorship amount: Rs. "))
if (venue < 0 or equipment < 0 or catering < 0
        or transport < 0 or certificate < 0
        or participants < 0 or fee < 0 or sponsorship < 0):
    print("Invalid input")
else:
    variable_ cost = catering + transport + certificate
    total_ cost = venue + equipment + (variable_ cost * participants)
    contribution = participants * fee
    available_ funds = contribution + sponsorship
    if available_ funds < total_ cost:
        shortfall = total_ cost - available_ funds
    else:
```

```
    shortfall = 0
if fee > variable_ cost:
    remaining_ cost = venue + equipment
    break_ even = int(remaining_ cost / (fee - variable_ cost))
    if remaining_ cost % (fee - variable_ cost) != 0:
        break_ even = break_ even + 1
    if breakeven < 0:
        breakeven = 0

elif  venue + equipment == 0:
    break_ even = 0
else:
    break_ even = -1
print("Total Event Cost: Rs.", round(total_ cost, 2))
print("Participant Contribution: Rs.", round(contribution, 2))
print("Sponsorship Amount: Rs.", round(sponsorship, 2))
print("Total Available Funds: Rs.", round(available_ funds, 2))
print("Sponsorship Shortfall: Rs.", round(shortfall, 2))
if break_ even == -1:
    print("Break-even is not possible with this registration fee.")
else:
    print("Minimum Break-even Registrations:", break_ even)
if available_ funds >= total_ cost:
    print("Status: Event Budget Covered")
else:
    print("Status: More Funds Req
```
**uired")**

**8.text cases**

| Test | Total cost | Participant contribution | Sponsorship | Expected result |
|---|---|---|---|---|
| 1 | ₹10,000 | ₹6,000 | ₹4,000 | Budget covered |
| 2 | ₹15,000 | ₹8,000 | ₹3,000 | ₹4,000 shortfall |
| 3 | ₹8,000 | ₹8,000 | ₹0 | Budget covered |

**9.sample output**

Campus Event Financial Planning

Enter venue cost: Rs. 5000

Enter equipment cost: Rs. 2000

Enter catering cost per participant: Rs. 100

Enter transport cost per participant: Rs. 50

Enter certificate cost per participant: Rs. 20

Enter number of participants: 100

Enter registration fee per participant: Rs. 200

Enter sponsorship amount: Rs. 5000

--- Event Financial Report ---

Total Event Cost: Rs. 24000.0

Participant Contribution: Rs. 20000.0

Sponsorship Amount: Rs. 5000.0

Total Available Funds: Rs. 25000.0

Sponsorship Shortfall: Rs. 0

Minimum Break-even Registrations: 44

Status: Event Budget Covered

**10.conclusion**

This project helps students understand how to plan a campus event budget using Python. It calculates the total cost, participant contributions, sponsorship

shortfall, and minimum registrations needed to break even. It uses basic concepts such as variables, arithmetic operators, conditional statements, and input/output functions.